DevCraft — Technical Brief

1. The Problem Statement

Order Intake for a Single-Operator Business

Build an **offline-first order management system** that converts unstructured customer messages into structured, queryable job records, and keeps them consistent across devices without a reliable network.

The user is a one-person business — a tailor, a tiffin service, an electrician, a home baker. Orders arrive as messy WhatsApp text in Hinglish. The operator has one mid-range Android phone, patchy connectivity, and no time. That context defines your constraints. It is not what you are being scored on.

You are being scored on the system you build.

There are several defensible architectures here and they will perform differently. That is the point of the event.

Objective 1 — Message Parsing

Convert a raw customer message into a structured order record.

Input: free text, Hinglish and Devanagari mixed, no fixed grammar.

> *"bhaiya 2 kurta chahiye navy blue, chest 40, parso tak ho jayega kya? last time jaisa hi"*

Required output schema:

json

```
{
  "customer": "string | null",
  "items": [{ "description": "string", "quantity": "integer", "attributes": {} }],
  "due_date": "ISO-8601 date | null",
  "amount": "number | null",
  "references_prior_order": "boolean",
  "confidence": "float 0-1",
  "needs_clarification": "boolean"}
```

Must handle:

- Hinglish, Devanagari, and Roman transliteration in the same message
- Relative and colloquial dates — *parso*, *agle mangalwar*, *is weekend*, *10 tarikh*
- Quantities written as words, digits, or implied
- Attributes without labels — a bare "40" that means chest size in context
- Ambiguity. A message that genuinely cannot be parsed must set `needs_clarification`, not guess. Confident wrong answers score worse than flagged uncertainty.

Approach is entirely open. Rules and regex, a fine-tuned small model, an LLM API with a local fallback, a hybrid pipeline — all valid. You will be measured on output, then asked to defend the choice.

Constraint: the parser must degrade gracefully with no network. If your primary path is a hosted API, you need a fallback that still produces a usable record offline. Systems that only work online will fail the offline test in Section 4.

Objective 2 — Offline-First Persistence

The application must be **fully functional with the network disabled**, from a cold start.

Requirements:

- Create, read, update and delete orders with zero connectivity
- State survives app termination and device restart
- Cold start to interactive in under 3 seconds on a mid-range Android device
- Initial download under 5 MB for web or PWA builds
- No blocking network call anywhere in the critical path

Objective 3 — Sync and Conflict Resolution

Two devices — the operator's phone and a tablet, or two staff — both go offline. Both edit the same order. Both reconnect.

Requirements:

- Convergence must be **deterministic**. The same sequence of offline edits must always produce the same final state, regardless of reconnection order.
- **No silent data loss.** If two edits genuinely conflict, either merge them or surface the conflict. Discarding one without telling anyone is a failure.
- Your strategy must be documented in the README and defensible under questioning.

Last-write-wins with timestamps, an operation log, CRDTs, or a custom merge policy are all acceptable. What is not acceptable is not having thought about it. This objective separates teams more than any other.

Objective 4 — Query Layer

The operator must be able to answer operational questions without scrolling:

- What is due today, and what is overdue?
- Which customers owe money, and how much in total?
- What did this customer order last time, and at what specification?
- What is my committed capacity this week?

Interface is your call — structured filters, natural-language query, voice. It must work offline.

Objective 5 — Deployment

The system must be **deployed and reachable by a judge on their own device**. A video is not a deliverable.

Any of the following is acceptable:

- Public URL — Vercel, Netlify, Render, Fly, anything
- Installable APK, sideloaded onto a judge's phone
- Self-hosted on your laptop, reachable over the venue network
- `docker-compose up` from your repo, run by a judge on the spot

Cloud hosting is not required. Local and self-hosted deployments are fully valid. What is required is that a judge can reach a running instance without your machine driving it.

---

2. What You Deliver

Deliverable	Detail
Deployed system	Reachable and operable by a judge. Working software, not a recording.
Source repository	Public GitHub link or shared with organisers. Commit history from kickoff.

README	Architecture overview, the parsing approach and why, the conflict resolution strategy, how to run it locally, known limitations.
Technical presentation	5 minutes on architecture and decisions, then 3 minutes of Q&A and a live judge-driven test.

The README carries real weight. Known limitations, stated honestly, score better than a README that pretends there are none.

---

3. Provided Materials

At kickoff, every team receives:

- **`messages_train.json`** — 250 labelled customer messages with expected structured output. Your development and validation set.
- **`schema.json`** — the exact output contract. Field names and types are fixed. Do not modify it.
- **`conflict_scenarios.md`** — three scripted offline-edit sequences your sync layer must handle.

Held back: a further 50 labelled messages, released only at judging.

---

4. Evaluation Protocol

Three of the five criteria are measured, not judged. Every team runs the same tests.

Test A — Parsing accuracy *(run live at judging)*

Judges provide `messages_test.json` — 50 held-out messages. You run them through your system in front of the panel and output results in the required schema. Scored automatically:

Measure	Weight within Test A
Field-level extraction accuracy	60%
Date resolution accuracy	20%
Correct `needs_clarification` flagging	20%

Your system must accept a batch file and emit results. Build that entry point early — a team that cannot run the test loses the entire criterion.

Test B — Offline behaviour *(judge-executed)*

1. Cold start with the device in airplane mode. Stopwatch to interactive.
2. Create, edit and delete an order fully offline.
3. Force-kill the app, restart, confirm state survived.
4. Restart the device, confirm state survived.

Test C — Conflict resolution *(judge-executed)*

Two devices, both offline, both edit the same order per the scripted scenarios. Reconnect in one order, then reset and reconnect in the reverse order.

Pass requires: identical final state both times, and no data lost without surfacing it.

---

5. Judging Criteria

Criterion	Weight	How it is assessed
Parsing accuracy	25	Test A, scored automatically on held-out data
Offline correctness and sync	25	Tests B and C, judge-executed, largely objective
Architecture and engineering decisions	20	Presentation and Q&A. Did they choose deliberately and can they defend it?
End-to-end completeness and deployment	20	Does the whole system work, deployed, in a judge's hands?
Query layer and interface	10	Does the operational layer answer real questions, offline?

Scoring notes for judges

- **Cap completeness at 10** if the system needs the team's own machine or a team member driving it.
- **Cap architecture at 10** if no member can explain a randomly chosen file.

- **A high accuracy score with no offline fallback is not a win.** A team hitting 92% via a hosted API that dies in airplane mode has not solved this problem statement. Weight Test B accordingly.
- **Do not reward scope.** Four objectives done properly beats five done halfway.
- **Reward stated limitations.** A team that says "our date parser fails on fiscal-quarter references and here's why" is more trustworthy than one claiming full coverage.

---

6. Rules on Pre-Built Code

Allowed: any framework, library, package or SDK; UI component libraries; public starter templates and boilerplate; open-source components under a permissive licence, credited in the README; pre-trained models from public repositories.

Not allowed: project-specific code written before kickoff; a pre-existing repository renamed; work carried over from a previous hackathon or personal project; a parser built before the problem statement was released.

Enforcement: repositories are initialised at kickoff and organisers record the first commit hash and timestamp. Commit history must show work distributed across the 24 hours. Judges may request `git log` at any point.

---

7. Rules on AI Usage

AI coding tools are fully permitted. Copilot, Claude, Cursor, agentic tools, anything. Use them as hard as you like.

One absolute condition:

> **Every team member must be able to explain any part of the codebase on demand.**

At judging, a judge opens a file at random and asks a member of their choosing — not yours — what it does, why it is structured that way, and what breaks if a specific line changes. "The AI wrote that" is a failing answer and caps your architecture score at 10.

This is a better filter than detection software. It is indifferent to how code was produced and tests only whether you understand what you are shipping.

Using an LLM inside your product is a legitimate architectural choice, not a violation — subject to the offline fallback requirement in Objective 1.